

© Brendan Choi 2021

B. Choi, *Introduction to Python Network Automation*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_9

9. Regular Expressions for Network Automation

Brendan Choi¹

(1) Sydney, NSW, Australia

This chapter is dedicated to learning regular expressions basics. Python provides native ways to process and dissect strings of text but requires many lines of code to locate the exact string. Although Python provides powerful string indexing methods, in real life you will face challenges each time you need to handle a large string. I find it fascinating that no other Python network automation books genuinely emphasize the importance of data wrangling (or massaging) with regular expressions. Many books sweep regular expressions under the rug as it is not the most exciting topic to discuss. I realized that I must master regular expressions to get anywhere with any programming language. Still, I also did not dare tackle it seriously at first. Only when I started working on real projects did I realize my ignorance over regular expressions. I understand that this chapter's content will be painful to digest for many readers, but you should stick with it and focus on completing this chapter; you will thank me later.



Regular expressions universally apply to all computer programming languages and are not unique to Python. Once you have mastered them, you can apply them to all programming languages such as Java, C++, JavaScript, Ruby, and Perl. In Python, the `re` module provides you with the passageway to regular expressions. By having a deep understanding of regular expressions and applying this knowledge to your Python programs via `re` modules, you will become a better Python coder. What you learn here will be applied to network automation application development in the labs. Mastering regular expressions is one topic no Python coder should skip. Although this is one of the shorter chapters in the book, you will gain valuable skills that will last for a lifetime.

After learning basic Python syntax and concepts, you will want to write Python applications fluently. As I will be mentioning throughout the book, writing code in Python is not all about the syntax and concepts; as with all things in computing, writing code has everything to do with proper data handling. This data could come from the user and application vari-

ables, reading files, scraping the Web, or logs between two computers. Writing code in any programming language almost always involves handling data. After some time writing Python code, I soon realized that it is impossible to develop any network automation application without understanding regular expressions in full.

If you are serious about Python coding, you must master the art of regular expressions and the `re` module. You will learn the basics of regex and apply regular expressions on real Cisco router and switch text files so you can relate the information to the real production examples.

For the chapter requirements, you will need access to your Windows 10 host PC and Ubuntu Server virtual machine running on VMware Workstation 15 Pro, as depicted in Figure 9-1.

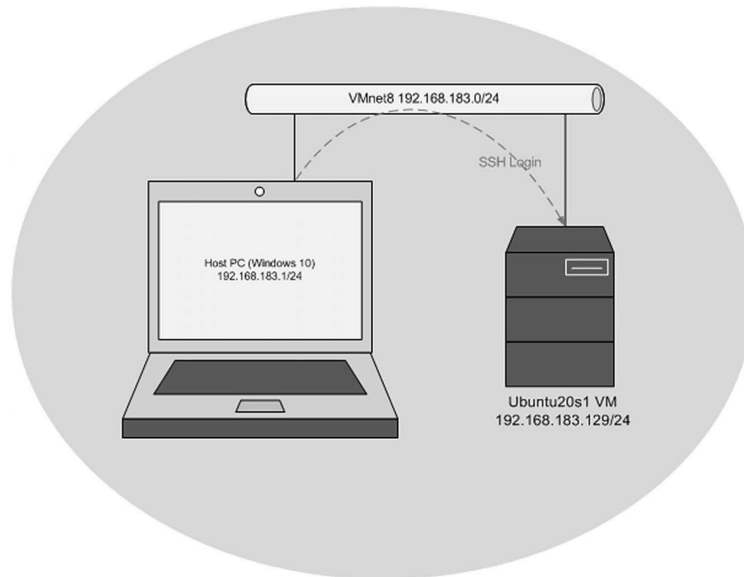


Figure 9-1. Devices required in Chapter 9

Why Regex?

text. A regular expression (*regex*) is a standard method used in computing to process complicated strings of text and is not exclusive to Python; it is used everywhere while performing data processing. At first glance, learning regular expressions can be seen as a distant subject to learning Python. Still, as you move to the intermediate level, the real power of regular expressions will come to light. In Python, the regular expression is also supported through a built-in Python module called the `re` module. Some readers may argue that learning regular expressions is not an essential part of learning Python, but as you work on real problems to solve automation problems, you will need to extract specific strings (data) from a source file or large strings. In coding, every comma, full stop, and white-space matters while handling data to create variables and run your applications to avoid data handling errors. If anyone asks me what programming languages such as C, C++, Java, Perl, Ruby, R, and Python do, without hesitation, my straightforward answer would be data processing. Every program written in a computer programming language must handle and process data to serve their purpose. In other words, we need a tool to improve our data handling and processing. As you have learned in Chapter

2, there are built-in methods to slice, concatenate, and index data in strings. When dealing with a more extensive set of data, you will run into many inconveniences as you might have to come up with your functions to handle big data better. The `re` module comes to the rescue as it is seen as a super-charged data handling method based on old regular expressions from the 1970s. Knowing when and where to use different Python modules can save you time from reinventing the wheel. First, you need to handle your data using the Python native methods. Second, explore data handling using the built-in `re` module. Third, explore even more advanced data handling methods using widely used Python libraries such as NumPy and Pandas. The NumPy library provides objects for multidimensional arrays, whereas Pandas provides an in-memory 2D table object called a *dataframe*. Every Python coder must learn regular expressions well and use them to their advantage to handle the data to extract a specific piece of string.

Another reason for including the topic of regex as a fully dedicated chapter was based on my recent experience while working on network automation. Regular expressions play a crucial role in coding, and I cannot emphasize enough their usefulness while working with Python code on various network automation projects. Once you reach an intermediate level in Python coding, inevitably you will confront data handling challenges, and if you refuse to learn how to use a regex, you will not progress. After following all exercises in this chapter, you will be well equipped with one of the most useful coding tools invented. The examples shown in this chapter may not be comprehensive enough for some, but enough scenarios are covered to help you get a grip on data handling through Python's `re` module. Every string match requirements are unique, so I urge you to go beyond this book and spend time researching more regex tutorials. There are plenty of free online training materials to practice use cases that apply to your data handling situations.

To re or Not to re

For starters, let's consider a straightforward example. You have applied for a job as a Python developer with a networking background. After you passed the first technical interview, the hiring manager has given you a take-home task to analyze a string of texts to be analyzed and transformed. Your task is to extract switch names and their MAC addresses to present the information in a specific format.

You have been given the following string. Your task is to print the switch name followed by the MAC address in 12 hexadecimal characters, where the first six digits (OUI) must be preserved to identify the manufacturer's ID. But the last six digits are masked with `*` to hide the real MAC address.

Here's a given string:

```
sw_mac = '''pynetauto-sw01 84:3d:c6:05:09:11
pynetauto-sw17 80:7f:f8:80:71:1b
pynetauto-sw05 f0:62:81:5a:53:cd'''
```

Also, any letters, including the hexadecimal a–f, must be capitalized, so the result must match the expected output shown here:

```
PYNETAUTO-SW01 843DC6*****
PYNETAUTO-SW17 807FF8*****
PYNETAUTO-SW05 F06281*****
```

If you have not studied regular expressions, you may take the following steps to write your program:

- 1.Remove colons and convert the string to capital letters.
- 2.Split the strings at the whitespace and add them to a list.
- 3.Remove whitespaces using the `strip()` method and make an updated list.
- 4.Use the `len()` method to identify names and MAC addresses based on the list's length of items. If the item has 14 characters, then it is a switch name; add to the `sw` list. If the item has 12 characters, then it is a MAC address; add it to the MAC list. During MAC appending, add `*****` to replace the second half of the MAC address.
- 5.Use Python dictionary's `zip` method to convert two lists into one dictionary.
- 6.Use the `for` loop to print the key and value from the dictionary's key-value pair.

In programming languages, they say, “There’s more than one way to skin a cat” but one of the ways (through code) that will achieve the desired result without using the `re` module will look similar to Listing 9-1.



Here’s a code and example string download tip: all the code presented in this book is available for download from `pynetauto` on GitHub. For the code used in this chapter, look for a zip file called `chapter9_codes.zip`.

URL: https://github.com/pynetauto/apress_pynetauto

```
>>> sw_mac = '''pynetauto-sw01 84:3d:c6:05:09:11
... pynetauto-sw17 80:7f:f8:80:71:1b
... pynetauto-sw05 f0:62:81:5a:53:cd'''
>>>
>>> sw_mac = sw_mac.replace(":", "").upper()           # 1
>>> sw_mac
'\nPYNETAUTO-SW01 843DC6050911 \nPYNETAUTO-SW17 807FF880711B \nPYNETAUTO-SW05 F062815A53CD\n'
>>> list1 = sw_mac.split(" ")                          # 2
>>> list1
['\nPYNETAUTO-SW01', '843DC6050911', '\nPYNETAUTO-SW17', '807FF880711B', '\nPYNETAUTO-SW05', 'F062815A53CD\n']
>>> list2 =                                             # 3
>>> for i in list1:                                     # 4
...     list2.append(i.strip())                        # 5
...
>>> list2
['PYNETAUTO-SW01', '843DC6050911', 'PYNETAUTO-SW17', '807FF880711B', 'PYNETAUTO-SW05', 'F062815A53CD']
>>>
>>> sw_list =                                           # 6
>>> mac_list =                                         # 7
>>> for i in list2:                                     # 8
```

```

...     if len(i) == 14:          # 9
...         sw_list.append(i)    # 10
...     if len(i) == 12:        # 11
...         i = i[:6] + "*****" # 12
...         mac_list.append(i)   # 13
...
>>> sw_list
['PYNETAUTO-SW01', 'PYNETAUTO-SW17', 'PYNETAUTO-SW05']
>>> mac_list
['843DC6*****', '807FF8*****', 'F06281*****']
>>> sw_mac_dict = dict(zip(sw_list, mac_list)) # 14
>>> for k,v in sw_mac_dict.items():           # 15
...     print(k, v)                          # 16 Lines
...
PYNETAUTO-SW01 843DC6*****
PYNETAUTO-SW17 807FF8*****
PYNETAUTO-SW05 F06281*****

```

Listing 9-1. Native Python Methods

You do not need to worry about or understand how to read the previous code. What is important is that you can count the number of code lines needed to achieve the expected result. Not counting the newline and data lines, exactly 16 code lines were required to print the result using only Python methods (without using any modules). Yes, it works, but it takes many lines of code to achieve the goal.

After you have learned how to use a regular expression, you can use Python's `re` module to do the same task, and the data manipulation becomes a breeze. The number of code lines drops, and it is easier to write and read. Count the number of code lines in Listing 9-2; exactly four code lines were needed to print out the expected result (not counting the newline and data lines). So, when you compare the number of code lines between the two listings, you can see significant savings in the number of code lines used. This was a simple example, but think about more complex tasks working with big data. It will take a lot of time and elbow grease to write lengthy custom code to achieve the end goal. Using the regular expression module in Python code, your application code will become more concise, easier to write, and easier to read.

```

>>> sw_mac = '''pynetauto-sw01 84:3d:c6:05:09:11
... pynetauto-sw17 80:7f:f8:80:71:1b
... pynetauto-sw05 f0:62:81:5a:53:cd'''
>>> import re # 1
>>> sw_mac = sw_mac.replace(":", "").upper() # 2
>>> pattern = re.compile("([0-9A-F]{6})" "([0-9A-F]{6})") # 3
>>> print(pattern.sub("\g<1>*****", sw_mac)) # 4
PYNETAUTO-SW01 843DC6*****
PYNETAUTO-SW17 807FF8*****
PYNETAUTO-SW05 F06281*****
>>>

```

Listing 9-2. `re` Example

There is no need to write the code yourself at this stage. To download and run the code, you can go to my `pynetauto` GitHub site and download the code in Listings 9-1 and 9-2.

Studying Regular Expressions Using Python

There are many ways you can study regular expressions with Python, and here we will discuss how you can study in a few different ways. Try a study method that suits you the best and stick to it until you complete the exercises in this chapter.



If you want to follow along with Listings [9-3](#) and [9-4](#), you can download the `sh_ver.txt` file from my GitHub site and save it as `sh_ver.txt` in your Linux directory.

URL: https://github.com/pynetauto/apress_pynetauto

Method 1: Using Notepad++

You already installed Notepad++ in Chapter [2](#), and it is a great tool to use to explore regular expressions. With Notepad++, there are two ways to study regular expressions, and depending on the size of the data or text, you can choose either approach. You can use a single Python file for a short string, as shown in Figure [9-2](#), and execute the script by using Ctrl+F6 or Ctrl+F5 from Notepad++.

```

1 import re
2
3 sh_ver = '''
4 -----
5 Technology      Technology-package      Technology-package
6                Current      Type      Next reboot
7 -----
8 appx            None            None            None
9 uc              uck9            RightToUse      uck9
10 security        None            None            None
11 ipbase          ipbasek9         Permanent      ipbasek9
12 -----
13 cisco ISR4351/K9 (2RU) processor with 1687137K/6147K bytes of memory.
14 Processor board ID FDO3132B224
15 3 Gigabit Ethernet interfaces
16 31 Serial interfaces
17 1 Channelized E1/PRI port
18 32768K bytes of non-volatile configuration memory.
19 4194304K bytes of physical memory.
20 3223551K bytes of flash memory at bootflash:.
21
22 Configuration register is 0x2102
23 '''
24
25 # Only match Cisco router model number from show version output.
26 rt_model = re.findall("[A-Z]{3}\d{4}[/]\w+", sh_ver)
27 my_router = rt_model[0]
28 print(my_router)
  
```

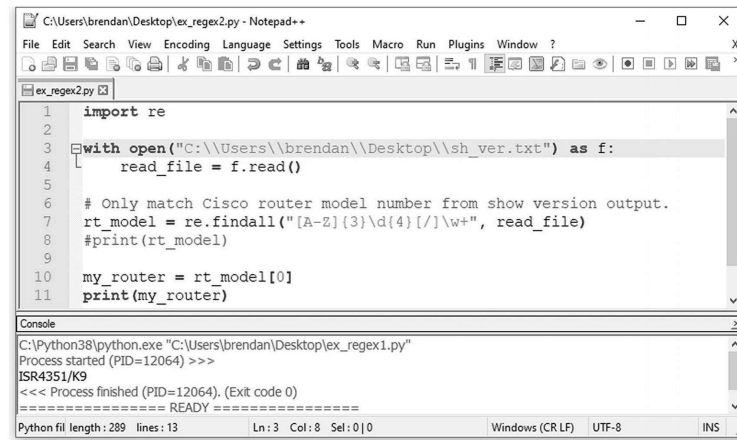
Console

```

C:\Python38\python.exe "C:\Users\brendan\Desktop\ex_regext1.py"
Process started (PID=11416) >>>
ISR4351/K9
<<< Process finished (PID=11416). (Exit code 0)
Python file length: 1,036 lines: 28   Ln: 7 Col: 67 Sel: 0|0   Windows (CR LF) UTF-8   INS
  
```

Figure 9-2. Studying regular expressions using Notepad++ with a single script

If the text data is lengthy or you have multiple files to read, you can use the file read method to read the files into your Python script and run the Python code you have written. This example is shown in Figures [9-3](#) and [9-4](#).



```

1 import re
2
3 with open("C:\\Users\\brendan\\Desktop\\sh_ver.txt") as f:
4     read_file = f.read()
5
6 # Only match Cisco router model number from show version output.
7 rt_model = re.findall("[A-Z]{3}\\d{4}[\\w+]", read_file)
8 #print(rt_model)
9
10 my_router = rt_model[0]
11 print(my_router)

```

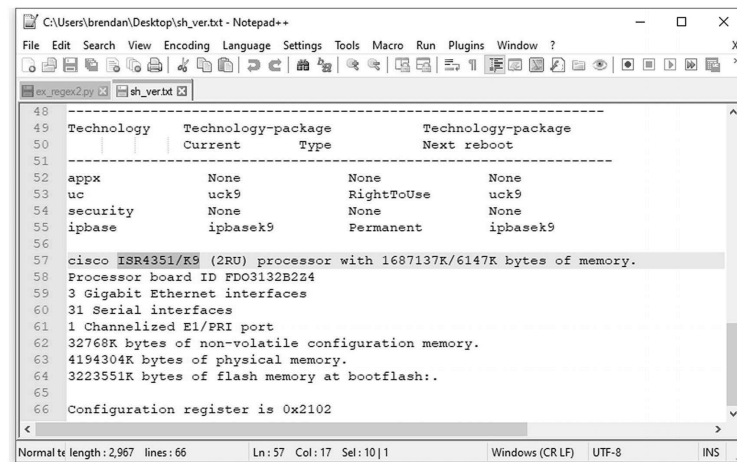
Console

```

C:\Python38\python.exe "C:\Users\brendan\Desktop\ex_regex1.py"
Process started (PID=12064) >>>
ISR4351/K9
<<< Process finished (PID=12064), (Exit code 0)
===== READY =====
Python file length: 289 lines: 13 Ln: 3 Col: 8 Sel: 0|0 Windows (CRLF) UTF-8 INS

```

Figure 9-3. Regular expression using Notepad++, read file method re script



```

48 -----
49 Technology Technology-package Technology-package
50           Current      Type      Next reboot
51 -----
52 appx      None          None          None
53 uc         uck9          RightToUse   uck9
54 security   None          None          None
55 ipbase     ipbasek9       Permanent   ipbasek9
56
57 cisco ISR4351/K9 (2RU) processor with 1687137K/6147K bytes of memory.
58 Processor board ID FDO3132B224
59 3 Gigabit Ethernet interfaces
60 31 Serial interfaces
61 1 Channelized E1/PRI port
62 32768K bytes of non-volatile configuration memory.
63 4194304K bytes of physical memory.
64 3223551K bytes of flash memory at bootflash:.
65
66 Configuration register is 0x2102

```

Normal file length: 2,967 lines: 66 Ln: 57 Col: 17 Sel: 10|1 Windows (CRLF) UTF-8 INS

Figure 9-4. Regular expression using Notepad++, read file method data file

Method 2: Using the Linux Shell

Like the Notepad++ method, you can run one of your Linux VM servers and SSH into the server to practice regular expressions on Python. If you want to test a simple regular expression, you can start an interactive session with the Python interpreter (see Listing 9-3). After you type the expression and press the Enter key, it immediately returns the result, or no result if your expression did not match any strings. You can also use this method using Python 3 on your Windows 10 host machine. As in Notepad++, you can also write a Python code first and run it from the terminal console to run the `re` match script, as shown in Listing 9-4. This method may sound difficult, but when you practice a couple of examples, you will get familiar with it, and using regular expressions with Python becomes easier.

```

pynetauto@ubuntu20s1:~$ pwd
/home/pynetauto
pynetauto@ubuntu20s1:~$ mkdir ex_regex
pynetauto@ubuntu20s1:~$ cd ex_regex
pynetauto@ubuntu20s1:~/ex_regex$ nano sh_ver.txt
pynetauto@ubuntu20s1:~/ex_regex$ ls
sh_ver.txt
pynetauto@ubuntu20s1:~/ex_regex
pynetauto@ubuntu20s1:~/ex_regex$ python3
Python 3.8.2 (default, Jul 16 2020, 14:00:26)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.

```

```
>>> import re
>>> with open("/home/pynetauto/ex_regex/sh_ver.txt") as f:
...     read_file = f.read()
...
>>> # Only match Cisco router model number from show version output.
>>> rt_model = re.findall("[A-Z]{3}\d{4}\w+", read_file)
>>> print(rt_model)
['ISR4351/K9']
>>> my_router = rt_model[0]
>>> my_router
'ISR4351/K9'
```

Listing 9-3. Regular Expression on Python Interpreter

```
pynetauto@ubuntu20s1:~/ex_regex$ pwd
/home/pynetauto/ex_regex
pynetauto@ubuntu20s1:~/ex_regex$ ls
sh_ver.txt
pynetauto@ubuntu20s1:~/ex_regex$ nano ex9.4_sh_ver.py
pynetauto@ubuntu20s1:~/ex_regex$ cat ex9.4_sh_ver.py
import re
with open("/home/pynetauto/ex_regex/sh_ver.txt") as f:
    read_file = f.read()
# Only match Cisco router model number from show version output.
rt_model = re.findall("[A-Z]{3}\d{4}\w+", read_file)
print(rt_model)
my_router = rt_model[0]
print(my_router)
pynetauto@ubuntu20s1:~/ex_regex$ python3 ex9.4_sh_ver.py
['ISR4351/K9']
ISR4351/K9
```

Listing 9-4. Regular Expression Writing Python Code on Linux

Regular Expression Breakdown: [A-Z]{3}\d{4}[/]\w+

[A-Z]{3}	\d{4}	[/]	\w+
Three capital letters	Four digits	Character /	Any string with at least one occurrence

Method 3: Using the Internet to Study Regular Expressions

One of the most effective ways to study regular expressions is to use a web browser and view content on the Internet. There are plenty of websites providing free regular expression exercises. Some sites are biased toward one programming language over another, but try to find programming language-neutral sites and practice more. Once you build your confidence in regular expressions, you can start using one of the first two Python `re` methods shown earlier. Practicing the `re` module in Python is important as that is the way you will be using a regular expression in Python. One of the better regular expression practice sites is regex101 (<https://regex101.com>). See Figure 9-5.

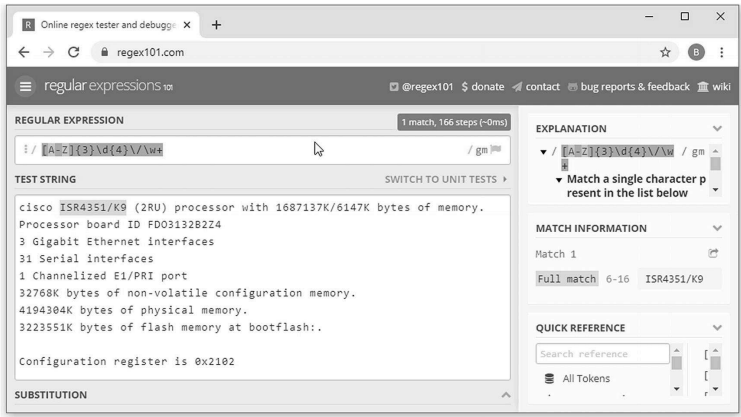


Figure 9-5. Regular expressions practice website

If you are on the go all the time but still want to get regular expression practice, you can practice on Android devices or Apple iOS devices. Download Regexpal (for Android phones) or RegEx Lab (for iOS devices) or similar apps on your mobile device. These applications are useful for on-the-go simple regular expression practice at any time, anywhere. See [Figure 9-6](#).

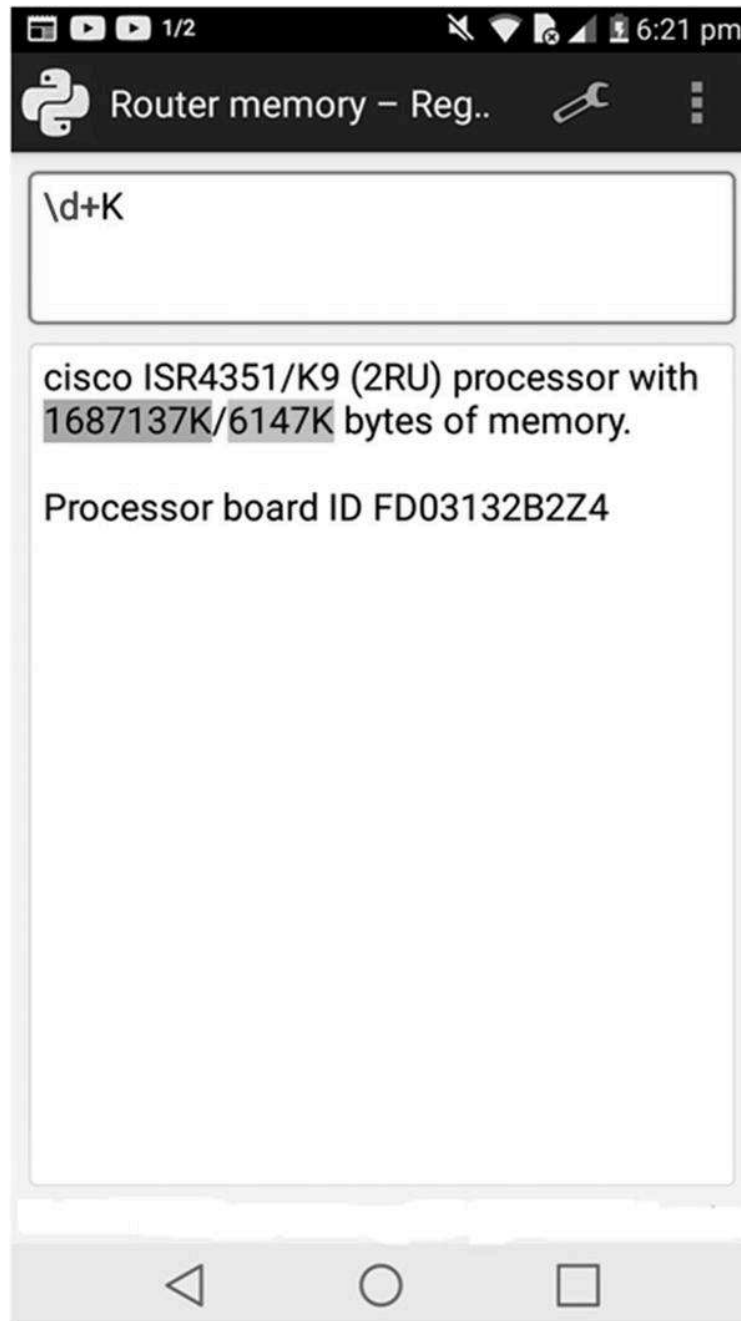


Figure 9-6. Android regular expression app example

Regex Operation: The Basics

Regular expressions use metacharacters to match static or dynamic data strings. The word *meta* is a Latin (Greek) word meaning *beyond*, *after*, or *behind*. When used in a programming language, metacharacters have special meaning from their real representation so that metacharacters can be defined as the characters having hidden meanings. The metacharacters include the following:

```
. ^ $ * + ? \ | ( ) [ ] { }
```

When one of these metacharacters is used in regular expressions, it carries some special meaning. The last six characters are in pairs of two, so the brackets must be used in pairs in the regular expressions. Let's start with the most straightforward regular expression and slowly build up our knowledge. Also, if you want to express these characters as a single literal character, then the

metacharacter can be enclosed in a set of a square bracket ([]) or escaped using the backslash (\). However, pay special attention to metacharacters ^ and \ ; these two characters cannot be literally matched using the square bracket method. See Table 9-1.

Table 9-1. Regular Expression Meatacharacters

Metacharacter	Terminology	Escape with \	Match Literally Using []
.	Dot	\.	[.]
^	Caret	\^	[\^]
\$	Dollar	\\$	[\$]
*	Star	*	[*]
+	Plus	\+	[+]
?	Question mark	\?	[?]
\	Backslash	\\	[\\]
	Pipe	\	[]
()	Round brackets	\(\)	[()]
[]	Square brackets	\[\]	[[]]
{ }	Curly brackets	\{ \}	[{ }]

To get around with the literal matching method with ^ and \ , you can add a backslash in front of the ^ and \ inside the square bracket. See Listing 9-5.



As all other chapters, to get a full understanding of regular expression concepts, you are expected to open your Python interpreter and type all the content marked in bold in each exercises. This includes everything after the three greater-than signs (>>>).

```
>>> import re
>>> expr = " . ^ $ * + ? \ | ( ) [ ] { }"
>>> re.search(r'\^', expr)
<re.Match object; span=(3, 4), match='^'>
>>> re.search(r'\\', expr)
<re.Match object; span=(13, 14), match='\'\'>
```

Listing 9-5. Matching Metacharacter ^ and \ Using Square Brackets ([])

If ^ or \ is used inside a square bracket set, you are required to add the escape backslash in front of the metacharacters. There are good reasons for this, and when used inside square brackets, ^ is used to negate the character following it. Type in Listing 9-6 into your Python interpreter and study it carefully. [^a]

would mean match all characters but *a*. In Listing 9-6, all characters are matched except the letter *a*.

```
>>> import re
>>> re.findall('[^a]', 'abracadabra')
['b', 'r', 'c', 'd', 'b', 'r']
```

Listing 9-6. Meaning of `[^a]`

The backslash inside the brackets, `[\\]`, also has a minor problem as the regular expression recognizes the backslash as a negating character for the closing square bracket, `]`. So, we have to negate the backslash by adding another backslash. Look at Listing 9-7, where the backslash had to be used to match backslash and a closing square bracket, `]`.

```
>>> re.search(r'[\\]', 'match \ or ']
<re.Match object; span=(6, 7), match='\\'>
>>> re.search(r'[\\]', 'match \ or ']
<re.Match object; span=(11, 12), match=']'>
```

Listing 9-7. Meaning of `[\\]`

Character Class (`[]`)

The first metacharacter we will learn is the character class metacharacter. Usually, a regular expression made up of character classes is expressed between the left open square bracket (`[`) and the right closed square bracket (`]`). All characters are allowed inside the character class, `[ ]`.

For example, a regular expression of `[aei]` will match any one of the vowel letters *a*, *e*, or *i*. Let’s look at the real use of words associated with alarms: *ding*, *buzz*, *beep*, *clang*.

Regular Expression	Words (String)	Matched	Explanation
[aei]	ding	ding	Matches the character <i>i</i> literally (case sensitive)
	buzz	buzz	No match; <i>buzz</i> does not contain <i>a</i> , <i>e</i> , or <i>i</i>
	beep	beep	Matches the character <i>e</i> twice
	clang	clang	Matches the character <i>a</i> literally

When a hyphen (`-`) is used inside the square brackets with letters or numbers, it abbreviates the expressions of a range. For example, `[a-z]` takes a meaning of all lowercase letters. `[A-Z]` takes a meaning of all uppercase letters from *A* to *Z*. You already have guessed it: to express all letters in both lowercase and uppercase, the regular expression to match becomes `[a-zA-Z]`. If you want to express a hexadecimal number, you will use `[0-9a-fA-F]`, meaning all whole numbers and both lower and uppercase let-

ters starting from *a* to *f*. In a regex, capitalization does matter. A common example of `[0-9]` takes the meaning of from 0 to 9, meaning `[0 1 2 3 4 5 6 7 8 9]`.

Almost all characters can be used between the square brackets, `[ ]`, but with a single exception of `^` (caret) symbol (`^` metacharacter). When `^` is used inside the square brackets, the expression used takes the opposite meaning or meaning of `NOT`. For example, if you want to use an expression to exclude any whole numbers, you can use `[^0-9]`. If you want not to match any uppercase letters, you can use `[^A-Z]`.

For commonly used regular expressions such as `[0-9]` and `[a-zA-Z]`, there are particular expressions to save time and make the regular expressions more compact and readable. These are tabulated here for your reference:

Regular Expression	Interchangeable Expression	Explanation
<code>\d</code>	<code>[0-9]</code>	Matches whole numbers from 0 to 9.
<code>\D</code>	<code>[^0-9]</code>	Matches all characters except whole numbers.
<code>\s</code>	<code>[\t\n\r\f\v]</code>	Matches all whitespace. A whitespace is included.
<code>\S</code>	<code>[^ \t\n\r\f\v]</code>	Matches all characters except whitespace characters.
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	Matches all alphanumeric letters.
<code>\W</code>	<code>[^a-zA-Z0-9_]</code>	Does not match alphanumeric letters. Matches all symbols such as <code>% # @</code> .

In the world of programming, the backslash (`\`) is commonly used to negate the meaning of the expression that comes after the backslash. For example, `d` is the letter *d* in the alphabet, but if it is combined with the backslash and becomes `\d`, this takes on a different meaning. Also, the particular expressions with capital letters always take the opposite meaning of the small, lettered expression.

Dot (.): Single Character Match

The regular expression `.` (dot) matches all characters except for line terminators such as `\n`. Interestingly, a regular expression provides a method to include `\n`, as we will see in an example later, but by using the `re.DOTALL` option, the `.` (dot) can also include the newline character `\n`.

Let’s consider the following regular expression example.

Regular Expression	Explanation
<code>d.g</code>	Matches any single character between the letters <i>d</i> and <i>g</i>
Any character specified must match. For example, the letter <i>d</i> must be matched in the first position, and the letter <i>g</i> must match in the third position. The middle character can match any character except the new character <code>\n</code> .	

Regular Expression	Words (String)	Matched	Explanation
<code>d.g</code>	<code>dog</code>	<code>dog</code>	Matches all characters <i>d</i> , <i>o</i> , and <i>g</i> .
	<code>d%g</code>	<code>d%g</code>	Matches all characters <i>d</i> , <i>%</i> sign, and <i>g</i> .
	<code>d\ng</code>	<code>None</code>	Not matched, <code> n</code> is ignored and not matched. <i>d</i> and <i>g</i> are matched, but <code> n</code> is ignored, so <i>d ng</i> is not matched.

What if the dot character is in between the square brackets, `[. ]`?

Regular Expression	Explanation
<code>d[.]g</code>	The dot character between the brackets takes its literal meaning as a single dot (<code>.</code>). So, it will only match <code>d.g</code> but not <code>dog</code> or <code>d%g</code> .

Asterisk (*): Repetition

Let’s zoom through this one and look at the following regular expression.

Regular Expression	Explanation
<code>zo*m</code>	Match if the character in front of the star occurs zero, one, or many times. The preceding expression is matched. In this case, the letter <i>o</i> .

The metacharacter `*` has the meaning of repetition, and it matches the preceding expression *o* zero, one, or almost infinite number of times. `*` becomes handy while trying to match unpredictable occurrences where a character may or may not appear; hence, it may or may not match.

All of the examples are matched by the `*` metacharacter `.`

Regular Expression	Words (String)	Matched	Explanation
zo*m	zm	zm	Matches <i>o</i> zero times. Even without <i>o</i> , <i>zm</i> is matched.
	zom	zom	Matches <i>o</i> once.
	zoom	zoom	Matches <i>oo</i> in the middle.

Plus (+): Repetition

A plus sign (+) is another metacharacter associated with the repetition. It is a little bit similar to the star but slightly different because zero times is not considered a match. With the use of the + metacharacter, at least one character must be matched. Let’s use the same character again for an explanation.

Regular Expression	Explanation
zo+m	Matches if the letter <i>o</i> appears one or more times

The + metacharacter matches some words.

Regular Expression	Words (String)	Matched	Explanation
zo+m	zm	None	No <i>o</i> in the middle, so not matched
	zom	zom	Matches <i>z</i> , <i>o</i> , and <i>m</i>
	zoom	zoom	Matches <i>z</i> , <i>oo</i> , and <i>m</i>

{m, n}: Repetition

Using the metacharacter {*m*, *n* }, you can match the number of repetitions. The letter *m* is the start of the match count, and *n* is the end of the match count. For example, o{1, 3} takes the meaning of matching the character *o* one to three times. Another example is o{3, }, which means that the repetition of the prepending character *o* must match at least three times or more. Another example is o{, 3} will match the same character *o* up to three times. So, the regular expression of {0, } is equivalent to + and {1, } is equivalent to *. Let’s study {*m*, *n*} by looking at some examples first.

{m}

The reference letter or *m* is the exact number of repetitions required to match the prepending character. Look at a simple example and explanation.

Regular Expression	Explanation
<code>zo{1}e</code>	Matches the first letter <code>z</code> , then matches the letter <code>o</code> <code>m</code> number of times (<code>m=1</code>), in this case once only, and then matches the letter <code>e</code> . So, the expected match letter is <code>zoe</code> . Note, the regular expressions are case sensitive.

Let’s look at more examples to help your understanding. If you can type these on your computer keyboard while following the book, you will learn a lot more.

Regular Expression {m}	Words (String)	Matched	Explanation
<code>o{2}</code>	<code>oo</code>	<code>oo</code>	Matches characters <code>oo</code>
	<code>zoo</code>	<code>zoo</code>	Matches characters <code>oo</code>
	<code>boom</code>	<code>boom</code>	Matches characters <code>oo</code>
<code>zo{2}m</code>	<code>zm</code>	<code>None</code>	Missing <code>oo</code> , so not matched
	<code>zom</code>	<code>None</code>	Missing <code>oo</code> , so not matched.
	<code>zoom</code>	<code>zoom</code>	Matches characters <code>z</code> , <code>oo</code> , and <code>m</code>

{m, n}

The first reference number, or `m`, is the minimum repetition number, and the second reference number, or `n`, is the maximum repetition number. First, a simple explanation of how `{m, n}` works .

Regular Expression	Explanation
<code>o{2, 5}</code>	Matches characters <code>oo</code> , <code>ooo</code> , <code>oooo</code> , or <code>ooooo</code> . Matches from two to five occurrences of the letter <code>o</code> .

Now let’s look at some simple examples.

Regular Expression {m, n}	Words (String)	Matched	Explanation
<code>zo{1, 3}m</code>	<code>zm</code>	<code>None</code>	Not matched, <code>o</code> matches 0 times

Regular Expression {m, n}	Words (String)	Matched	Explanation
	zoom	zoom	Matches <i>z</i> , <i>oo</i> , and <i>m</i>
	zooom	None	Not matched, one too many <i>o</i>

? (Question Mark: Repetition)

The metacharacter uses the same meaning as {0, 1} and can be used interchangeably. Again using a simple example, here is more explanation of how ? works in a regular expression.

Regular Expression	Explanation
Zoe?	Matches <i>Zo</i> or <i>Zoe</i> . The character before the question mark, which is <i>e</i> , becomes optional. So, even the string is <i>Zo</i> . The regular expression will still match the word. This can be written as <code>Zoe{0, 1}</code> .

Use the same example as before to better understand the ?.

Regular Expression	Words (string)	Matched	Explanation
zo?m	zm	zm	Matches <i>z</i> and <i>m</i> . Letter <i>o</i> is optional, so it does not have to match.
	zom	zom	Matches <i>z</i> , <i>o</i> , and <i>m</i> .
	zoom	None	Not matched as only expecting one <i>o</i> .

As shown earlier, *, +, and ? metacharacters can be replaced by {m, n} methods, but using *, +, and ? makes regular expression easier to read and understand, so whenever possible, try to use *, +, and ? instead of {m, n} format.

Python’s re Module

As discussed earlier, Python provides the regular expression feature out of the box, and it is supported by a standard module called re (regular expression). This module will be pre-installed as part of the standard Python library when Python is installed on your OS.

To begin your journey into the Python re module , it is worth mentioning that there is some difference in the ways or styles you can write your

Python code using the `re` module. You can choose to keep your script structure simple, standard, or structured. In a simple style, you can write the regular expression statement in a single line of code, as shown in the simple style example in Table 9-2. When you use the standard style, you do not have to use a `re.compiler` statement, but you can still add some style by separating the string and regular expression statement. If you choose to have a little more consistency in your code, you can use the compiler style, using the `re.compiler` statement. Any coding style will work fine, but the advantage of using a `re.compiler` statement in your code is obvious. It gives a little bit more control and structure to your code and hence also adds styling consistency.

Let’s compare three styles in questions quickly by looking at each example. All three examples will return the same result but written in two different styles.

Table 9-2. Different Ways to Use re Module in Python

Style	Example
Simple	<pre>import re m = re.findall('\dx\d{4}', "Configuration reg- ister is 0x2102") print(m)</pre>
Standard	<pre>import re expr = "Configuration register is 0x2102" m = re.findall('\dx\d{4}', expr) print(m)</pre>
Compiler	<pre>import re expr = "Configuration register is 0x2102" ← expr for expression p = re.compile('\dx\d{4}') ← p for pattern m = p.findall(expr) ← m for match print(m)</pre>
Result	<pre>['0x2102']</pre>

In the third compiler style, we have given a separate variable `p` to the compiler statement, which contains the regular expression we want to match. And then the variable `m` performs the match function using variables `expr` and `p`. If the compiled regular expression has to be used more than two times, it would be advantageous to use this method over the simpler methods. You have to note that Python allows you to write short-codes, but you lose the structure and style, whereas more code lines give you better control and structure, but you have to write more lines of code to achieve the same result. In my honest opinion, there are no right or wrong answers here; this all depends on your style and personal preference.

Python re String Methods

There are four basic types of regular expression search methods we have to master. Let’s quickly go over the four methods and then review each by doing some exercises on the Python interpreter. See Table [9-3](#).

Table 9-3. re String Methods

Method	Explanation
<code>re.match()</code>	Search the regular expression pattern on the first line and return the match object. (Only searches the first line.) If a match is not found, returns <code>None</code> .
<code>re.search()</code>	Search the regular expression pattern and return the first occurrence. Unlike the <code>re.match()</code> method, this method will check all lines. If a match is not found, returns <code>None</code> .
<code>re.findall()</code>	Search the regular expression pattern and match all occurrences. Unlike the <code>re.match()</code> or <code>re.search()</code> method, this method returns all the nonoverlapping matches of patterns in a string as a list of strings.
<code>re.finditer()</code>	Search the regular expression pattern, and this method returns an iterator yielding <code>MatchObject</code> instances over all nonoverlapping matches for the string’s <code>re</code> pattern.

re.match()

Listing [9-8](#), Listing [9-9](#), and Listing [9-10](#) will return the same matched objective but are written differently. Carefully study how each line is written differently. Again, there is no right or wrong answer here but different ways to write the same code to achieve the same objective.

```
>>> import re
>>> re.match(r'\d\w\d{4}', '0x2142 Configuration register is 0x2102')
<re.Match object; span=(0, 6), match='0x2142'>
Listing 9-8. re.match() method 1
```

```
>>> import re
>>> expr = '0x2142 Configuration register is 0x2102'
>>> re.match(r'\d\w\d{4}', expr)
<re.Match object; span=(0, 6), match='0x2142'>
Listing 9-9. re.match() method 2
```

```
>>> import re
>>> expr = '0x2142 Configuration register is 0x2102'
>>> p = re.compile(r'\d\w\d{4}')
>>> m = p.match(expr)
>>> print(m)
<re.Match object; span=(0, 6), match='0x2142'>
Listing 9-10. re.match() method 3
```

Let's do another match exercise. In Listing 9-11, the `match` object is returned with the span containing the `match` string's start and end positions. By contrast, if there is no matched object in Listing 9-12, the returned value is `None`. In Python, `None` serves the same purpose as `null` in other languages, meaning that it did not match anything.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "five regular expression"
>>> m = p.match(expr)
>>> print(m)
<re.Match object; span=(0, 4), match='five'>
Listing 9-11. re.match()
```

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "5 regular expression"
>>> m = p.match(expr)
>>> print(m)
Result: None
Listing 9-12. re.match()
```

The previous exercises are essential in the way we structure our Python regular expression script flow. Looking at the returned values, it could be a best practice to structure our Python `re` scripts in the following format, so the script continues to run only if a match is found:

Recommended script flow for regular expression for this chapter

```
import re
p = re.compile("Enter_re_here")
expr = 'string_to_search_here'
m = p.match(expr)
if m:
    print('Match found: ',
          m.group())
else:
    print('Match not found')
```

← import re module
← compile a pattern
← data or expression
← matching patterns using pattern
← execute only if a match is found

`re.search()`

As shown earlier, let's use the `re.search()` method to perform the match function, and in Listing 9-13, the returned result is the same as `re.match()` where the regular expression matched the string and returned the matched object.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "five regular expression"
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(0, 4), match='five'>
Listing 9-13. re.search()
```

On the other hand, in Listing 9-14, unlike the `re.match()` method, the `re.search()` method regular expression did not match digit 5, so it jumped to the next match and returned the word *months*, which matched the regular expression. So, the `match()` method is typically used if searching from the beginning of the string, whereas the `search()` method can be used to match the first instance, searching the whole string.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "5 regular expression"
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(2, 9), match='regular'>
```

Listing 9-14. re.search()

re.findall()

This time let's get some practice with the `findall()` method using the same regular expression flow. Enter each line into your Python Interpreter console to get more practice. Unlike the previous two methods, the `findall()` method returns the matched objects as a list. In Listing 9-15, each word has been returned as strings in a list.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "five regular expression"
>>> m = p.findall(expr)
>>> print(m)
['five', 'regular', 'expression']
```

Listing 9-15. re.findall() exercise 1

In Listing 9-16, `findall()` matches all strings satisfying the regular expression condition but ignores the digit 5. This method will try to match and return all matched objects in a list format.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "5 regular expression"
>>> m = p.findall(expr)
>>> print(m)
['regular', 'expression']
```

Listing 9-16. re.findall() exercise 2

re.finditer()

The last exercise is on the `finditer()` method. As before, let's type in the code and learn by doing some exercises (see Listing 9-17 and Listing 9-18). The `finditer()` method returns the same result as the `finditer()` method, but it returns an iterator over all nonoverlapping matches for the string's regular expression pattern. It might be a powerful tool for text processing, but the use cases for the `finditer()` method are very narrow.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "five regular expression"
>>> m = p.finditer(expr)
```

```
>>> print(m)
<callable_iterator object at 0x000001E581F1B5E0>
>>> for r in m:
...     print(r)
...
<re.Match object; span=(0, 4), match='five'>
<re.Match object; span=(5, 12), match='regular'>
<re.Match object; span=(13, 23), match='expression'>
Listing 9-17. re.finditer() exercise 1
```

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "5 regular expression"
>>> m = p.finditer(expr)
>>> print(m)
<callable_iterator object at 0x000001E581F1B5E0>
>>> for r in m: print(r)
...
<re.Match object; span=(2, 9), match='regular'>
<re.Match object; span=(10, 20), match='expression'>
Listing 9-18. re.finditer() exercise 2
```

Match Object Method

Earlier, we saw the returned objects from regular expression matches using match and search methods, but we still had some questions about the matched strings and exactly how they work. To better understand the attributes of these returned objects, after running the match and search methods, we can use match object methods to answer some of our questions. First, let’s quickly review [Table 9-4](#).

Table 9-4. re Match Object Method

Match Method	Explanation
group()	Returns matched string
start()	Returns the start position of the matched string
end()	Returns the end position of the matched string
span()	Returns the start and end position of matched string in tuple format

Now let’s learn match object methods through exercises to confirm the returned objects from the matched returned objects. In the first exercise with the match method, we can individually get the match object attributes using group, start, end, and span match methods. Enter the code in [Listing 9-19](#) into the interpreter and check the result interactively.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "automation"
>>> m = p.match(expr)
>>> print(m)
<re.Match object; span=(0, 10), match='automation'>
```

```
>>> m.group()
'automation'
>>> m.start()
0
>>> m.end()
10
>>> m.span()
(0, 10)
```

Listing 9-19. re.match()

In the search match method shown in Listing [9-20](#), if we want to find out the exact match object location in the memory, we can issue `group`, `start`, `end`, and `span` without the round bracket set at the end. Anyway, we should focus more on the `group()`, `start()`, `end()`, and `span()` result of the match object attributes.

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> expr = "5. regular expression"
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(3, 10), match='regular'>
>>> m.group
<built-in method group of re.Match object at 0x000001E581F00C00>
>>> m.group()
'regular'
>>> m.start
<built-in method start of re.Match object at 0x000001E581F00C00>
>>> m.start()
3
>>> m.end
<built-in method end of re.Match object at 0x000001E581F00C00>
>>> m.end()
10
>>> m.span
<built-in method span of re.Match object at 0x000001E581F00C00>
>>> m.span()
(3, 10)
```

Listing 9-20. re.search()

Compile Options

When you compile regular expressions, you can also add options to your expressions. First, quickly review the options. When using these regular expression options, you can use the full descriptive option names like `re.DOTALL`, `re.IGNORECASE`, `re.MULTILINE`, and `re.VERBOSE` or use the abbreviated version such as `re.S`, `re.I`, `re.M`, and `re.X`. See Table [9-5](#).

Table 9-5. re Compile Options

Options	Abbreviation	Explanation
<code>DOTALL</code>	<code>S</code>	Matches any character including a newline, <code>\n</code> .
<code>IGNORECASE</code>	<code>I</code>	Makes the regex case-insensitive or ignore case. All major regex engines match in case-sensitive mode; I mode disables case sensitivity.

Options	Abbreviation	Explanation
MULTILINE	M	^ and \$ will match the start and end of a line, instead of the whole string. It enables the regular expression engine to handle an input string that consists of multiple lines.
VERBOSE	X	Allows the use of verbose mode. Whitespace is ignored. Spaces, tabs, and carriage returns are not matched as spaces, tabs, and carriage returns.

Once again, no pain, no gain, so let’s type the following exercises into the Python Interpreter and learn from the examples.

re.DOTALL (re.S)

The DOTALL option is used to include a newline in the match string, and let’s check how the DOTALL option works in real use. See Listing 9-21 and Listing 9-22.

```
>>> import re
>>> expr = 'a\nb'
>>> p = re.compile('a.b')
>>> m = p.match(expr)
>>> print(m)
None
```

← No match found due to newline

Listing 9-21. Without re.DOTALL()

```
>>> import re
>>> expr = 'a\nb'
>>> p = re.compile('a.b', re.DOTALL)
>>> m = p.match(expr)
>>> print(m)
<re.Match object; span=(0, 3), match='a\nb'> ← Matches object
```

Listing 9-22. re.DOTALL()

In Listing 9-21, \n literally represents a newline, and for this reason, the regular expression a.b alone cannot match a\nb. In Listing 9-22, when we enable the DOTALL option using re.DOTALL, we can confirm that even the \n character is matched as part of the string. Under normal circumstances, you will not be using the DOTALL option to match the newline character. Instead, it will be used to match strings in multiple lines by ignoring the newline characters.

re.IGNORECASE (re.I)

When the IGNORECASE option is enabled, it makes the regular expression case insensitive so both uppercase and lowercase letters are matched, disregarding the casing of alphabet characters. Again, let’s do some exercises to see how re.IGNORECASE (re.I) works in action. See Listing 9-23.

```

>>> import re
>>> expr1 = 'automation'
>>> expr2 = 'Automation'
>>> expr3 = 'AUTOMATION'
>>> p = re.compile('[a-z]+', re.IGNORECASE)
>>> m1 = p.match(expr1)
>>> print(m1)
<re.Match object; span=(0, 10), match='automation'>
>>> m2 = p.match(expr2)
>>> print(m2)
<re.Match object; span=(0, 10), match='Automation'>
>>> m3 = p.match(expr3)
>>> print(m3)
<re.Match object; span=(0, 10), match='AUTOMATION'>

```

Listing 9-23. re.IGNORECASE()

Only a single exercise is enough to comprehend the use of `re.IGNORECASE` or `re.I` use. In Listing 9-23, the same pattern has been used three times to match the patterns in three different expressions (strings). Three casing types (all lowercase, first-letter capitalization, and all uppercasing expressions) are used as our target strings. The regular expression will match all three strings as the `re.I` option was enabled in the regular expression pattern.

re.MULTILINE (re.M)

The `re.MULTILINE` or `re.M` option is used with `^` and `$` regular expression metacharacters. As explained before, the `^` metacharacter is used to indicate the start of a string, and `$` is used to mark the end of a string. In layman's term, a regular expression of `^Network` means the first string must begin with the word *network*, whereas `automation$` means the last string must end with the word *automation*. Once again, let's learn `MULTILINE` by doing. See Listing 9-24 and Listing 9-25.

```

>>> import re
>>> expr = '''Regular Engineers
... Regular Network Engineers
... Regular but not so regular Engineers'''
>>> p = re.compile('^R[w+\S]')
>>> m = p.findall(expr)
>>> print(m)
['Regular']

```

Listing 9-24. without re.MULTILINE()

```

>>> import re
>>> expr = '''Regular Engineers
... Regular Network Engineers
... Regular but not so Regular Engineers'''
>>> p = re.compile('^R[w+\S]', re.MULTILINE)
>>> m = p.findall(expr)
>>> print(m)
['Regular', 'Regular', 'Regular']

```

Listing 9-25. ^ and re.MULTILINE()

In both exercises, the data (expression) consists of three lines starting with the common word `Regular`. Each line begins with the same word,

and the regular expression `^R\w+\s` matches the first word only, starting with a capital `R` followed by alphanumeric letters to nonwhitespace (`\s`). In Listing 9-24, without `re.MULTILINE` option, the search only matches the first occurrence of the word, `Regular`. Whereas in Listing 9-25, with `re.MULTILINE` option enabled, the search matches the word, `Regular` at the start of a new line on the multiple lines.

re.VERBOSE (re.X)



Try to type as much as you can and as accurately as you can. If you need to cross-check your typos, you can download the exercises used in this chapter from my download site. You can optionally copy and paste the information for your exercise as required.

URL: https://github.com/pynetauto/apress_pynetauto/

Most of the regular expression examples introduced in this chapter have been basic compared to the regular expressions used in real, production-ready scripts.

Now, look at the following two examples (Listing 9-26 and Listing 9-27); nevertheless, they are the same scripts returning the same value. For novice Python coders with limited exposure to regular expressions, the compiled regular expression in the first example (`r'[1-9](?:\d{0,2})(?:,\d{3})*(?:\.\d*[1-9])?|0?\.\d*[1-9]|0'`) will look like nothing more than gibberish. The compiled regular expression seems somewhat over-complicated. It is used to match the digits related to the dates and purposefully matches digits or digits with a comma used in the year. The first example will be what we are familiar with thus far, but the second example takes full advantage of the regular expression `re.VERBOSE` option to add comments and explain how the regular expression works. Although both return the same result, the latter increases the readability, perhaps even for the person who has written this regular expression. With the `re.VERBOSE` or `re.X` option, you can decode and add optional comments to add readability.

```
import re
expr = 'I was born in 2,009 and I am 15 years old. I started my primary school in 2,010'
p = re.compile(r'[1-9](?:\d{0,2})(?:,\d{3})*(?:\.\d*[1-9])?|0?\.\d*[1-9]|0')
m = p.findall(expr)
print(m)
```

Listing 9-26. Without re.VERBOSE

```
import re
expr = 'I was born in 2,009 and I am 15 years old. I started my primary school in 2,010'
p = re.compile(r"""
[1-9]          # Match a single digit between 1-9
(?:\d{0,2})    # Match digit equal to [0-9] between 0 and 2 times
(?:,\d{3})*    # Match the character ,(Comma) literally, match a digit equal to [0-9] \
               exactly 3 times) Zero and unlimited times
(?:\.\d*[1-9])? # Match the character. (Dot) literally, match a digit equal to [0-9] \
               zero and unlimited times, match a single digit between [1-9] \
               zero and one time.
|
# OR
0?\.\d*[1-9]   # Match 0 zero or one time, match . (Dot) literally, match a digit \
               equal to [0-9] zero or unlimited times, and match a digit between [1-9]
```

```
|          # OR
0          # Match one 0
""" , re.VERBOSE)
m = p.findall(expr)
print(m)
Result: ['2,009', '15', '2,010']
```

Listing 9-27. With re.VERBOSE

\: Confusing Backslash Character

\ in Python gives special meaning to a metacharacter; it is an escape character in both Python strings and for the regex engine, to which you are eventually passing your patterns. When the slash character is used in the Python `re` engine, this can cause some confusion around how many escape characters you have to use to match the exact string you want to extract from your data. To remove confusion, when compiling a regular expression in Python, `r` (raw string notation) is appended immediately before the regular expression. Let's type in the following two examples (Listing 9-28 and Listing 9-29), the first without the use of raw string notation, `r`, and the second using the raw string notation, `r`. In the following examples, we are trying to match the string `\scored`.

```
>>> import re
>>> expr = 'Our team \scored three goals\\'
>>> p1 = re.compile('\scored')
>>> p2 = re.compile('\scored')
>>> p3 = re.compile('\\scored')
>>> p4 = re.compile '\\scored')
>>> p5 = re.compile '\\\\scored')
>>> print(p1.findall(expr))
[]
>>> print(p2.findall(expr))
[]
>>> print(p3.findall(expr))
['\scored']
>>> print(p4.findall(expr))
['\scored']
>>> print(p5.findall(expr))
[]
```

Listing 9-28. Backslashes Without Raw String Notation

In Listing 9-29, the compiled regular expression with three backslashes and four backslashes matched and returned the same result. This can be confusing as you might be unsure whether you have to use three or four backslashes to match the word with the literal backslash character, `\scored`.

```
>>> import re
>>> expr = 'Our team \scored three goals\\'
>>> p1 = re.compile(r'\scored')
>>> p2 = re.compile(r'\scored')
>>> p3 = re.compile(r '\\scored')
>>> p4 = re.compile(r '\\\\scored')
>>> print(p1.findall(expr))
[]
>>> print(p2.findall(expr))
['\scored']
>>> print(p3.findall(expr))
[]
>>> print(p4.findall(expr))
```

[]

Listing 9-29. Backslash with Raw String Notation

In Listing [9-30](#), we are using the raw string notation, `r`, and it is clear that the result indicates that you have to use two backslashes to match the target string `\scored`. Using one, three, and four backslashes with raw string notation will not match the target string.

```
>>> import re
>>> expr = 'Our team \scored three goals\\'
>>> p2 = re.compile(r'\\scored')
>>> m = p2.findall(expr)
>>> print(m)
['\\scored']
>>> n = m[0]
>>> n
'\\scored'
>>> for x in n:
...     print(x, end=")
...
\\scored
```

Listing 9-30. Backslash with Raw String Notation

In Listing [9-30](#), the raw string match method is used, and in this example, the raw string match method matches both `\`'s (backslashes) and returns the matched string, `\\scored` in a list. To convert an item in a list, you can use the indexing method.

Regular Expressions: A Little Revision Plus More

In this section, let's quickly review what we have learned so far and cover a few more metacharacters and regular expression methods such as grouping, lookahead search, etc.

More Metacharacters

We have covered the most commonly used metacharacters so far, but we have to cover a few more common metacharacters to move forward. The characteristics of the metacharacters we discuss here are a little different from the ones discussed earlier in this chapter. We have already studied metacharacters such as `+`, `*`, `[ ]`, and `{ }`. These metacharacters will only search the strings once as they change their positions in a matching string. Let's now extend our metacharacter vocabulary by introducing a few more new ones, and then we will look at the grouping and lookahead and lookbehind examples.

OR Operator (|)

In a regular expression, the `|` (pipe) metacharacter has the same meaning as `or`. The regular expression of `a|b` has a similar meaning to `[ab]`, but they operate differently. Both `|` and `[ ]` are OR operators, and they will try to match specified characters in the string, but the matched and returned results are a little different. Let's look at the following examples and precisely understand the real difference between `|` and `[ ]`. See Listing [9-31](#) and Listing [9-32](#).

```
>>> import re
```

```
>>> re.findall('a[bc]', 'a, ab, ac, abc, acb, ad')
['ab', 'ac', 'ab', 'ac']
Listing 9-31. a[bc]
```

```
>>> re.findall('a(b|c)', 'a, ab, ac, abc, acb, ad')
['b', 'c', 'b', 'c']
Listing 9-32. a(b|c)
```

In Listing [9-31](#), you have used the `[ ]` regular expression to match `b` or `c`; the regular expression match's returned values also contain the leading `a`. But in Listing [9-32](#), you replaced `[ ]` with `|`, and the returned values are only the matched values of either `b` or `c` but no leading `a`. So, the difference between `[ ]` and `|` is the way it matches the leading character(s), but one returns the leading character(s), and one doesn't.

Now, looking at Listing [9-33](#) and Listing [9-34](#), this is illustrated with a regular expression's range option, and it becomes more apparent that one prints the leading character and one doesn't.

```
>>> re.findall('3[a-f]', '3, 3a, 3c, 3f, 3g')
['3a', '3c', '3f']
Listing 9-33. 3[a-f]
```

```
>>> re.findall('3(a|b|c|d|e|f)', '3, 3a, 3c, 3f, 3g')
['a', 'c', 'f']
Listing 9-34. 3(a|b|c|d|e|f)
```

Listing [9-35](#) and Listing [9-36](#) are simple examples of `|` (OR) with the words *apple* and *raspberry*. In Listing [9-36](#), `re.findall` match method matches both words, *raspberry* and *apple*. The matched words are returned as a list.

```
>>> re.match('apple|raspberry', 'raspberry pie')
<re.Match object; span=(0, 8), match='raspberry'>
Listing 9-35. apple|raspberry
```

```
>>> print(re.findall('apple|raspberry', 'raspberry and apple pie'))
['raspberry', 'apple']
Listing 9-36. apple|raspberry
```

^ and \$ (Anchors)

`^` (caret) will look for a match beginning at the first character of a string. When you use it with the `re.MULTILINE` or `re.M` option, you enable the regular expression at each newline of your data or strings. Let's quickly do some exercises using your Python interpreter. Listing [9-37](#) and Listing [9-38](#).

```
>>> re.findall('^Start', 'Start to finish')
['Start']
Listing 9-37. ^Start
```

In Listing [9-38](#), `^` instructs the regular expression `^Start` to match the word *start* at the start of the string.

```
>>> re.findall('finish$', 'Start to finish')
['finish']
```

Listing 9-38. finish\$

In Listing [9-38](#), \$ instructs the regular expression to match the word *finish* at the end of the string.

In Listing [9-39](#), combining both ^ and \$, the regular expression matches the exact string starting with *S* and end with *sh*.

```
>>> re.findall('^S.+sh$', 'Start to finish')
['Start to finish']
```

Listing 9-39. ^S.+sh\$

```
>>> re.findall('^S.+sh$', 'Start to finish\nSpecial fish\nSuper fresh', re.MULTILINE)
['Start to finish', 'Special fish', 'Super fresh']
```

Listing 9-41. ^S.+sh\$ and re.M

In Listing [9-40](#), using the `re.M` module, you have matched three lines of strings starting with *S* and ending with *sh*.

Now let's take a look at a simplified practical example through another exercise. The code is available for download from my GitHub website, and the filename is `5.7.1.2_5.py` for this exercise

(https://github.com/pynetauto/apress_pynetauto). See Listing [9-41](#).

```
>>> import re
>>> expr = '''SYDCBDPIT-ST01#sh ip int brief
... Interface          IP-Address      OK? Method Status      Protocol
... Vlan1              unassigned     YES NVRAM  up          up
... Vlan50             10.50.50.11    YES NVRAM  up          up
... FastEthernet0      unassigned     YES NVRAM  down        down
... GigabitEthernet1/0/1 unassigned     YES unset  down        down
... GigabitEthernet1/0/2 unassigned     YES unset  up          up
... GigabitEthernet1/0/3 unassigned     YES unset  up          up
... '''
>>> p = re.compile('^Gig.+down$', re.MULTILINE)
>>> m = p.findall(expr)
>>> print(m)
['GigabitEthernet1/0/1 unassigned YES unset down down']
```

Listing 9-41. ^Gig.+up\$ and re.M

You can easily apply this to the real scenario where you want to extract some interface information from your networking device `show com-`mands. In Listing [9-41](#), we use the first few lines of Cisco switch's `show ip interface brief` commands to demonstrate this. Using the regular expression `^Gig.+up$`, we can quickly match the interfaces that have down status. For demonstration purposes, the example has been oversimplified. Still, in the real environment where you have to manage hundreds and thousands of switchport interfaces, you need the power of regular expression to process the collected data.

`\A and \Z`

In a single-line string match, `^` and `\A` work the same way, but the behavior differs while trying to match strings with multiple lines (see Listing [9-42](#) and Listing [9-43](#)). The same applies to `$` and `\Z`; the matching behaviors change while trying to match strings with multiple lines (see Listing [9-44](#)). When `re.M` or `re.MULTILINE` option is enabled, `^` can match at the start of strings and after each line break, but `\A` only ever matches at the start of the string (see Listing [9-45](#)). Also, `$` can match at the end of the string and before each line break (see Listing [9-46](#)), but `\Z` only ever matches at the end of the string (see Listing [9-47](#)).

```
>>> re.findall('^S.+sh', 'Start to finish')
['Start to finish']
Listing 9-42. ^S.+sh
```

```
>>> re.findall('\AS.+sh', 'Start to finish')
['Start to finish']
Listing 9-43. \AS.+sh
```

```
>>> re.findall('^S.+sh', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Start to finish', 'Super special fish', 'Super fresh fish', 'Super smelly fish']
Listing 9-44. ^S.+sh with re.MULTILINE
```

```
>>> re.findall('\AS.+sh', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Start to finish']
Listing 9-45. \AS.+sh with re.MULTILINE
```

```
>>> re.findall('S.+sh$', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Start to finish', 'Super special fish', 'Super fresh fish', 'Super smelly fish']
Listing 9-46. S.+sh$ with re.MULTILINE
```

```
>>> re.findall('S.+sh\Z', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Super smelly fish']
Listing 9-47. S.+sh\Z with re.MULTILINE
```

The following exercises demonstrate how mixed use of `^` or `\A` and `$` or `\Z` affects the matched return results. We artificially add newlines by separating each line with `\n` to create the newline effect. Come up with your examples and test these metacharacters to extend your understanding. See Listing [9-48](#), Listing [9-49](#), Listing [9-50](#), and Listing [9-51](#).

```
>>> re.findall('^S.+sh$', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Start to finish', 'Super special fish', 'Super fresh fish', 'Super smelly fish']
Listing 9-48. ^S.+sh$ with re.M
```

```
>>> re.findall('\AS.+sh$', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Start to finish']
Listing 9-49. \AS.+sh$ with re.M
```

```
>>> re.findall('^S.+sh\Z', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish', re.M)
['Super smelly fish']
Listing 9-50. ^S.+sh\Z with re.M
```

```
>>> re.findall('\AS.+sh\Z', 'Start to finish\nSuper special fish\nSuper fresh fish\nSuper smelly fish',
[]
```

Listing 9-51. \AS.+sh\Z with re.M

\b and \B

In a regular expression, `\b` represents a word boundary. In a sentence, the whitespacing between the leading word and the following word is represented by the usual word boundaries.

Word Boundary	Description
---------------	-------------

<code>\b</code>	Represents an anchor like <code>^</code> (similar to <code>\$</code> and <code>^</code>) matching positions where one side is a word character (like <code>\w</code>), and the other side is not a word character; for example, it may be the beginning of the string or a space character.
-----------------	---

<code>\B</code>	It comes with its negation, <code>\B</code> . This matches all positions where <code>\b</code> does not match and could be if we want to find a search pattern surrounded by word characters.
-----------------	---

According to regular expression literal rules, `\b` represents a backspace; hence, when you use it in a regular expression, you always have to use the raw string notation `r` to specify that this is not a backspace character.

In Listing 9-52, you are trying to match the word `computers` in the expression. In the pattern compiled, we have used the leading `\b` and ending `\b` as the word we are trying to match leading whitespace and ending with whitespace, so the word matches and returns the result.

```
>>> import re
>>> expr = "Small computers include smartphones."
>>> p = re.compile(r'\bcomputers\b')
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(6, 15), match='computers'>
```

Listing 9-52. `\b(word)\b` matched

In Listing 9-53, the word `microcomputers` has the word `computers` but starts with `Micro`, so the whitespace cannot match the word `computers` trailing the word `Micro`. The result is `None` as expected.

```
>>> import re
>>> expr = "Microcomputers include smartphones."
>>> p = re.compile(r'\bcomputers\b')
>>> m = p.search(expr)
>>> print(m)
None
```

Listing 9-53. `\b(word)\b` not match

In Listing 9-54, the first `\b` was dropped in our regular expression compiler, and now the match target word, *computers*, is matched and returns the expected result.

```
>>> import re
>>> expr = "Microcomputers include smartphones."
>>> p = re.compile(r'computers\b')
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(5, 14), match='computers'>
```

Listing 9-54. (word)\b matched

`\B` has an opposite meaning to `\b` and hence can be used to achieve the reverse of `\b`. In Listing 9-55, you try to match the word *computer* without the plural *s* at the end. So, use the leading `\B` and ending `\B` to match the word with pinpoint accuracy.

```
>>> import re
>>> expr = "Microcomputers include smartphones."
>>> p = re.compile(r'\Bcomputer\B')
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(5, 13), match='computer'>
```

Listing 9-55. \B(word)\B matched

Grouping

Say you have a string consisting of up and down status from a flapping link on a router, and you want to perform a match to search for continuous up status. After searching and matching the continuous up status, you can use the `group()` option to print the output (Listing 9-56). To compile a group in a regular expression, you have to use `()`. This is an impractical example only to introduce you to the concept of regular expression grouping. The next example will be more within the real context.

```
>>> import re
>>> expr = "downupupupdowndownupdowndown"
>>> p = re.compile("(up)+")
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(4, 10), match='upupup'>
>>> print(m.group(0))
upupup
```

Listing 9-56. Grouping exercise 1

Now let's consider a technical assistant center's country name and the fully qualified number for the United States. First, we want to compile a regular expression to match the whole string, starting with the country name and then followed by the technical assistant center telephone number. See Listing 9-57.

```
>>> import re
>>> expr = "United States 1 408 526 1234"
>>> p = re.compile(r"\w+\s\w+\s\d\s\d{3}\s\d{3}\s\d+")
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(0, 28), match='United States 1 408 526 1234'>
```

Listing 9-57. Grouping exercise 2

In Listing 9-57, you have used a mix of alphanumeric shorthand character `\w`, whitespaces shorthand character `\s`, and digit shorthand character `\d` with repetition `{n}` to match all characters. If we stop the string matching here, the data captured in this example is not well utilized. By using grouping, we can extract specific information from the matched strings. Say we want to extract only the country name (see Listing 9-58).

```
>>> import re
>>> expr = "United States 1 408 526 1234"
>>> p = re.compile(r"(\w+\s\w+)\s\d?\s\d{3}\s\d{3}\s\d+")
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(0, 28), match='United States 1 408 526 1234'>
>>> country = m.group(1)
>>> country
'United States'
```

Listing 9-58. Grouping exercise 3

In Listing 9-58, we can use the `group(1)` method to separate the information from the whole matched string, and now we can use this data as a variable or data in our script. Now let's quickly review the group method index to understand how the regular expression groupings are numbered. See Table 9-6.

Table 9-6. Group Method Index Meanings

Group Index	Explanation
<code>group(0)</code>	Whole matched string
<code>group(1)</code>	Matched first group
<code>group(2)</code>	Matched second group
<code>group(3)</code>	Matched third group
<code>group(n)</code>	Matched n^{th} group

The grouping method indexing is better explained using practical examples, as shown in Listing 9-59.

```
>>> import re
>>> expr = "United States 1 408 526 1234"
>>> p = re.compile(r"(\w+\s\w+)\s(\d?\s\d{3}\s\d{3}\s\d+)")
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(0, 28), match='United States 1 408 526 1234'>
>>> phone_number = m.group(2)
>>> phone_number
'1 408 526 1234'
```

Listing 9-59. Grouping Method Indexing exercise 1

In Listing 9-60, the `group(1)` is `(\w+\s\w+)`, which is the same `group(1)` from Listing 9-59 and `(\d?\s\d{3}\s\d{3}\s\d+)` becomes the `group(2)`, which captures the telephone number. See Figure 9-7.

```
>>> import re
>>> expr = "United States 1 408 526 1234"
>>> p = re.compile(r"(\w+\s\w+)\s((\d?)\s(\d{3})\s(\d{3})\s\d+)")
>>> m = p.search(expr)
>> m.group(0)
'United States 1 408 526 1234'
>>> m.group(1)
'United States'
>>> m.group(2)
'1 408 526 1234'
>>> m.group(3)
'1'
>>> m.group(4)
'408'
>>> m.group(5)
'526 1234'
```

Listing 9-60. Grouping Method Indexing exercise 2

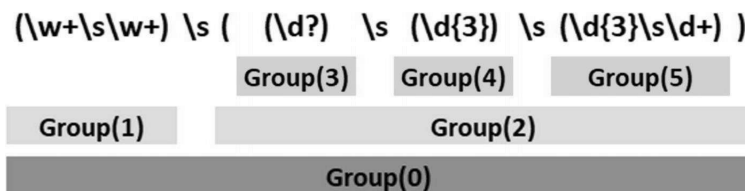


Figure 9-7. Group index in regular expression example

In Listing 9-61, the groups have been broken down into even smaller groups, and now we can separate the country code 1, area code 408, and local number 526 1234 from each other. Imagine all the possible things we can do with regular expression groupings in real networking automation scenarios. Imagine what you can do with the captured data from real production routers and switches and control your devices based on the collected and analyzed data.

```
>>> import re
>>> expr = "Did you know that that 'that', that that person used in that sentence, is wrong."
>>> p = re.compile(r'(\bthat)\s+\1')
>>> m = p.search(expr)
>>> print(m)
<re.Match object; span=(13, 22), match='that that'>
>>> m = p.search(expr).group()
>>> print(m)
that that
```

Listing 9-61. Referencing Grouped String

Another advantage of using group indexing is that you can use shorthand like `\1` to re-reference the first group. In the regular expression of `(\bthat)\s+\1`, the `+` has the meaning of matching the same strings as the previous group, and `\1` has the meaning of re-referencing group number 1. If there is a second group and you want to reference it, `\2` can be used.

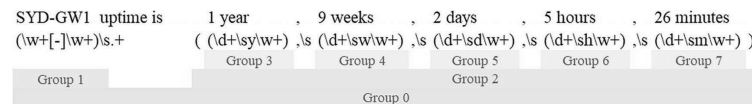
Regular Expression Named Groups

Let's assume that we are working on a project and writing a regular expression to process the data and fully utilizing the `re` grouping methods.

We have to work with a line extracted from a router's `show version` command, and it contains the router name and uptime. We want to use regular expression groups to extract the router name, the precise number of years, weeks, days, hours, and minutes the device has been running.

To match them as each group, one example of the grouping that will work will look similar to the regular expression. Still, since there are multiple groupings and you are reading somebody else's code, you would probably feel overwhelmed with just a single regular expression line.

After carefully studying the regular expression for 10 minutes, you have finally concluded that this is how the strings are matched and grouped. Unfortunately, with so many groups, you can easily get lost in translation. See Figure 9-8.



```
>>> import re
>>> expr = "SYD-GW1 uptime is 1 year, 9 weeks, 2 days, 5 hours, 26 minutes"
>>> p = re.compile(r'(\w+[-]\w+)\s.+((\d+\sy\w+),\s(\d+\sw\w+),\s(\d+\sd\w+),\s(\d+\sh\w+),\s(\d+\sm\w+))')
>>> m = p.search(expr)
>>> print(m.group(0))
SYD-GW1 uptime is 1 year, 9 weeks, 2 days, 5 hours, 26 minutes
>>> print(m.group(1))
SYD-GW1
>>> print(m.group(2))
1 year, 9 weeks, 2 days, 5 hours, 26 minutes
>>> print(m.group(3))
1 year
[... omitted for brevity]
```

```
>>> print(m.group(7))
26 minutes
```

Listing 9-62. Multiple Numbered Groups exercise

To make such an over convoluted use of grouping methods in the regular expression and add flexibility, regular expressions have a handy named group. You can simply add `?P<group_name>` at the beginning of the group and refer to the group as the group name you have specified. If we have to give group names to the previous example, it will look similar to Listing 9-63. Giving a name to a group gives more meaning to the data it matches, and you do not have to spend much time decoding the regular expression.

```
>>> import re
>>> expr = "SYD-GW1 uptime is 1 year, 9 weeks, 2 days, 5 hours, 26 minutes"
>>> p_named = re.compile(r'(?P<hostname>\w+[-]\w+)\s.+(?P<uptime>(?P<years>\d+\sy\w+),\s(?P<weeks>\d+\s
>>> m = p_named.search(expr)
>>> print(m.group("minutes"))
26 minutes
[... omitted for brevity]
>>> print(m.group("uptime"))
1 year, 9 weeks, 2 days, 5 hours, 26 minutes
>>> print(m.group("hostname"))
SYD-GW1
```

Listing 9-63. Named Group exercise

When you analyze the groups named above against the string data, the groupings will look like Figure 9-9.

SYD-GW1	uptime is	1	9	2	5	26
(\w+[-]\w+)	\s.+(	(\d+\sy\w+)	,\s	(\d+\sw\w+)	,\s	(\d+\sd\w+)
hostname	uptime	years	weeks	days	hours	minutes

Group 0

Figure 9-9. Regular expression named groups example

As you can see, the names relate to the information you are trying to match, and there is very little confusion coming from the multiple groupings. Rather than trying to decode someone else's cryptic regular expressions, you should be spending the valuable work hours on more mission-critical issues or providing good customer support experiences.

Lookahead and Lookbehind Assertions

Before diving into this section, I have to be up front with you. If you are a first-time regular expression apprentice, you may not understand the lookaheads and lookbehinds immediately. To ease the pain, I have used exercises and examples, but you may have to come back to this topic a few times before getting a full grasp of this concept.

Lookahead, Lookbehind, and Noncapturing group

Lookaheads and lookbehinds are known as *lookarounds* and often confuse novice regex learners. At first glance, lookaheads and lookbehinds look very confusing as they all start with a `?`, and the difference comes from the second and third metacharacter use. But once mastered well, they are handy tools to shorten our regular expressions. You do not need to comprehend this section straightaway; you can take your time to learn

the lookarounds. Also, use this section as a reference point for future reference. In lookarounds, there are four lookarounds and a noncapturing group, which all start with the `?` mark.

As a quick reminder before getting our hands dirty, Table 9-7 shows quick lookarounds and simple examples.

Table 9-7. The re Lookahead, Lookbehind, and Noncapturing Groups

Lookaround	Name	String	Example	Explanation
<code>?=</code>	Lookahead	abc	<code>a (?=b)</code>	Asserts that the regex must be matched
<code>?!</code>	Negative lookahead	abc	<code>a (?!b)</code>	Asserts that it is impossible to match the regex
<code>?<=</code>	Lookbehind	abc	<code>(?<=a) b</code>	Asserts that the regex must be matched
<code>?<!</code>	Negative lookbehind	abc	<code>(?<!a) b</code>	Asserts that it is impossible to match the regex
<code>?:</code>	Noncapturing group	abc	<code>a(?:b)</code>	Regex inside the parentheses must be matched but doesn't create the capturing group

Now go ahead and start typing in the text marked in bold in your Python interpreter and see how the returned objects differ from one another. In the following exercises, you will be using an abbreviated print function with a regular expression and the target string `'abc'`. The explanation is included next to each exercise.

Exercise #	Exercises	Explanation
1	<pre>>>> print(re.search('a(?=b)', 'abc')) <re.Match object; span=(0, 1), match="a"> >>> print(re.search('a(?=b)', 'xbc'))</pre>	Matches a as the next letter is b. No match as the first letter is not a before letter b.

Exercise #	Exercises	Explanation
	None	
2	<pre> >>> print(re.search('a(?!b)', 'abc')) None >>> print(re.search('a(?!b)', 'acd')) <re.Match object; span=(0, 1), match="a"> </pre>	No match as the next letter is b. Matches a as the next letter is not b (it is c).
3	<pre> >>> print(re.search('(=?a)b', 'abc')) <re.Match object; span=(1, 2), match="b"> >>> print(re.search('(=?a)b', 'xabc')) None </pre>	Matches b as the previous letter is a. No match as the previous letter is not a.
4	<pre> >>> print(re.search('(<!a)b', 'abc')) None >>> print(re.search('(<!a)b', 'xabc')) <re.Match object; span=(1, 2), match="b"> </pre>	No match as the previous letter is a. Matches b as the previous letter is not a.
5	<pre> >>> print(re.search('a(?:b)', 'abc')) <re.Match object; span=(0, 2), match="ab"> >>> print(re.search('a(=?b)', 'abc')) <re.Match object; span=(0, 1), match="a"> </pre>	Matches a and b, because b is followed by a. Both ab are matched. Matches a as the next letter is b. (This is the same as exercise 1.)
6	<pre> >>> import re >>> expr = "+1 408 526 1234" >>> p = re.compile(r"((?: (\+1) [-])?(?(\d{3}))\)?[-]?(\d{3})[-]?(\d{4}))") >>> m = p.search(expr) >>> print(m) </pre>	?: represents noncapturing group, Without ?:, +1 and the trailing space are captured as a group. With ?:, the regular expression will not make a group by the match inside the parentheses like it

Exercise #	Exercises	Explanation
	<pre><re.Match object; span=(0, 15), match='+1 408 526 1234'> >>> print(m.group(2)) +1</pre>	would otherwise do (typically, parentheses create a group). In other words, when you use <code>?:</code> , the group is matched but is not captured for backreferencing, which is why it's called a <i>non-capturing</i> group.

 Go to the RexEgg regex site for more deep dives and regular expression examples. This is one of my favorite sites to learn about regular expressions.

URL: www.rexegg.com/regex-lookarounds.html

Practice More Lookarounds

You need a lot of practice to wrap your head around this topic. This time, let's use the string 'abba' to make the exercise more exciting and use the full compile method to structure our learning.

```
expr = 'abba'
```

Please open the Python interpreter and enter the text marked in bold next to the three greater-than signs. Reading through this section will not help you retain the information, so you must do all exercises, from 1 to 10.

Exercise #	Exercise	Regex	Explanation
1	<pre>>>> import re >>> expr = 'abba' >>> p = re.compile('a(?=b)') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(0, 1), match="a"></pre>	<code>a(?=b)</code>	Find the a, which has b after it.
2	<pre>>>> import re >>> expr = 'abba' >>> p = re.compile('b(=?b)') >>> m = p.search(expr) >>> print(m)</pre>	<code>b(=?b)</code>	Find the first b, which has b after it.

Exercise #	Exercise	Regex	Explanation
	<pre> <re.Match object; span=(1, 2), match="b"> </pre>		
3	<pre> >>> import re >>> expr = 'abba' >>> p = re.compile('a(?!b)') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(3, 4), match="a"> </pre>	a(?!b)	Find the second a, which does not have b after it.
4	<pre> >>> import re >>> expr = 'abba' >>> p = re.compile('b(?!b)') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(2, 3), match="b"> </pre>	b(?!b)	Find the second b, which does not have b after it.
5	<pre> >>> import re >>> expr = 'abba' >>> p = re.compile('(?(<=a)b') >>> print(m) <re.Match object; span=(1, 2), match="b"> </pre>	(? <=a)b	Find the first b, which has a before it.
6	<pre> >>> import re >>> expr = 'abba' >>> p = re.compile('(?(<=b)b') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(2, 3), match="b"> </pre>	(? <=b)b	Find the second b, which has b before it.

Exercise #	Exercise	Regex	Explanation
7	<pre>>>> import re >>> expr = 'abba' >>> p = re.compile('(? <!a)b') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(2, 3), match="b"></pre>	<pre>(? <!a)b</pre>	Find the second b, which does not have a before it.
8	<pre>>>> import re >>> expr = 'abba' >>> p = re.compile('(? <!b)b') >>> m = p.search(expr) >>> print(m) <re.Match object; span=(1, 2), match="b"></pre>	<pre>(? <!b)b</pre>	Find the first b, which does not have b before it.
9	<pre>>>> import re >>> expr = 'abc' >>> p1 = re.compile('a(?:b)') >>> m1 = p1.search(expr) >>> print(m1) <re.Match object; span=(0, 2), match="ab"></pre>	<pre>a(?:b)</pre>	
10	<pre>>>> p2 = re.compile('a(?:=b)') >>> m2 = p2.search(expr) >>> print(m2) <re.Match object; span=(0, 1), match="a"></pre>	<pre>a(?:=b)</pre>	

Lookaround Application Examples

For current and future engineers studying the Python programming language, there is a lot of ground to cover. Until you start working on real projects at work, you may not see real value and power of regular expressions. So far, you have been introduced to some basic regular expressions

using exercises, and now you are gradually skilling up on regular expressions. However, unless you live and breathe regex every day on your job, it will take a long time to get to the top level. However, you can still learn the basics from this chapter and supplement with other materials found on the Internet and other books to get to the intermediary and advanced levels.

Let's use Cisco TAC's URL <http://www.cisco.com/techsupport> to practice what we have learned. See Listing 9-64.

```
>>> import re
>>> m = (re.search(r"\w{4,5}(?=:)", "http://www.cisco.com/techsupport"))
>>> print(m.group())
http
```

Listing 9-64. Only Use Lookahead Method to Print http

In Listing 9-64, the regular expression `\w{4,5}(?=:)` matches the previous alphanumerics 4 to 5 times which occurs in front (lookahead) of the colon (:).

```
>>> import re
>>> m = (re.search(r"(?<=\/)\w.+[\.]\w+[\.]\w+(?=\/)", "http://www.cisco.com/techsupport"))
>>> print(m.group())
www.cisco.com
```

Listing 9-65. Only Use Lookahead and Lookbehind Method to Print www.cisco.com

To match www.cisco.com only, we have used a lookbehind method (`?<=`), which starts with the forward slash, /, and then matched the `www` format. We then used a lookahead (`?=`) ending with /. Using lookarounds, you can match the strings quickly and more precisely.

The following expression can match any filenames that have the format of the `filename.file_extension`, such as `vlan.dat`, `sw1.bak`, `config.text`, and even a Cisco switch IOS name such as `2960x-universalk9-mz.152-2.E6.bin`. In the following example, we will compare the negative lookahead method compared to the `^`, the negation method.

```
.*[.].*$
```

In the expr (expression or string), take special care that `\n` has been used between filenames to create a newline effect in a single string. This is to simplify our exercise and save pages. The explanations are followed after the lines of each exercise.

In Listing 9-66, you will be using regular expression `.*[.].*$` to match all files in expr.

```
>>> import re
>>> expr = "vlan.dat\nsw1.bak\nconfig.text\npynetauto.dat\nsw1_old.bak\n2960x-universalk9-mz.152-2.E6.b
>>> m = re.findall(".*[.].*$", expr, re.M)
>>> m
['vlan.dat', 'sw1.bak', 'config.text', 'pynetauto.dat', 'sw1_old.bak', '2960x-universalk9-mz.152-2.E6.b
```

Listing 9-66. Match All File Types Using Regular Expression - `.*[.].*$`

In Listing [9-67](#) the `^` (caret) method is used to negate any files ending with the `.dat` extension and returns all the matched file names in a list. Make a special note that we are using the `re.findall` and `re.M` (MULTILINE) methods that we have learned earlier in this chapter.

In Listing [9-68](#) negative lookahead example returns the same result as Listing [9-67](#). There's more than one way to extract the same data.

```
>>> import re
>>> expr = "vlan.dat\nsw1.bak\nconfig.text\npynetauto.dat\nsw1_old.bak\n2960x-universalk9-mz.152-2.E6.b"
>>> m = re.findall(r".*[^d].*$", expr, re.M)
>>> m
['sw1.bak', 'config.text', 'sw1_old.bak', '2960x-universalk9-mz.152-2.E6.bin']
```

Listing 9-67. Filter files with file extensions not starting with the letter d (Not using lookahead method)

```
>>> import re
>>> expr = "vlan.dat\nsw1.bak\nconfig.text\npynetauto.dat\nsw1_old.bak\n2960x-universalk9-mz.152-2.E6.b"
>>> m = re.findall(r".*[^d](?!dat$).*$", expr, re.M)
>>> m
['sw1.bak', 'config.text', 'sw1_old.bak', '2960x-universalk9-mz.152-2.E6.bin']
```

Listing 9-68. Filter files with file extensions not starting with the letter d (using lookahead method)

In Listing [9-69](#) and Listing [9-6687](#), you have negated any filenames ending with the `.dat` or `.bak` using both negation methods. Still, the latter method has an advantage if the file extension extensions begin with the same letter or the length of the extension starts to vary.

```
>>> import re
>>> expr = "vlan.dat\nsw1.bak\nconfig.text\npynetauto.dat\nsw1_old.bak\n2960x-universalk9-mz.152-2.E6.b"
>>> m = re.findall(r".*[^d|b].*$", expr, re.M)
>>> m
['config.text', '2960x-universalk9-mz.152-2.E6.bin']
```

Listing 9-69. Filter Any Files Ending with `.dat` and `.bak` Without Using Lookaround Method

```
>>> import re
>>> expr = "vlan.dat\nsw1.bak\nconfig.text\npynetauto.dat\nsw1_old.bak\n2960x-universalk9-mz.152-2.E6.b"
> m = re.findall(r".*[^d](?!dat$|bak$).*$", expr, re.M)
>>> m
['config.text', '2960x-universalk9-mz.152-2.E6.bin']
```

Listing 9-70. Filter Any Files Ending with `dat` and `bak` Using Lookaround Method

Let's check this in Listing [9-71](#).

```
>>> import re
>>> expr = "file1.bak\nfile2.dat\nfile3.bakup\nfile4.data"
>>> m = re.findall(r".*[^d|b].*$", expr, re.M)
>>> m
[]
```

Listing 9-71. Filter Any Files Ending with `dat` or `bak` Using `^` Negation Method

In Listing [9-71](#) and Listing [9-72](#), the returned result is different as the regular expression used in Listing [9-71](#) filtered or negated all files, but in Listing [9-72](#), only the files ending with `.dat` or `.bak` files have been filtered. If we try to write a working regular expression without the negative lookahead method in Listing [9-72](#), the regular expression will be lengthy and cryptic. In Listing [9-72](#), using the

negative lookahead method, you can simply append the exact extension name with the | (OR) sign in the regular expression.

```
>>> import re
>>> expr = "file1.bak\nfile2.dat\nfile3.bak\nfile4.data"
>>> m = re.findall(r".*[\.](?!dat$|bak$).*$", expr, re.M)
>>> m
['file3.bak', 'file4.data']
```

Listing 9-72. Filter Any Files Ending with dat or bak Using Negative Lookahead Method

sub Method: Substituting Strings

Using the `sub` method in the regular expression, you can change or swap out matched strings. Let's do the first exercise and review the result.

Substitute Strings Using `sub`

As always, let's start with an exercise first and then review what you have learned from the exercise. See [Listing 9-73](#).

```
>>> import re
>>> p = re.compile('HP|Juniper|Arista')
>>> p.sub('Cisco', 'Juniper router, HP switch, Arista AP and Palo Alto firewall')
'Cisco router, Cisco switch, Cisco AP and Palo Alto firewall'
```

Listing 9-73. Use `sub` to substitute multiple matching words

In [Listing 9-73](#), using the `sub` method, “Juniper router, HP switch, Arista AP and Palo Alto firewall” was changed to “Cisco router, Cisco switch, Cisco AP and Palo Alto firewall.” Since Palo Alto was not in the compiled regular expression, only Juniper and HP were substituted with Cisco.

Follow through to [Listing 9-74](#), what if we only want to control the number of replacement?

```
>>> p.sub('Cisco', 'Juniper router, HP switch, Arista AP and Palo Alto firewall', count=1)
'Cisco router, HP switch, Arista AP and Palo Alto firewall'
```

Listing 9-74. Use `sub` to substitute a matching word only once

You can control the substitutions using the `count` parameter. In the previous example, `count 1` was used, so in [Listing 9-74](#), only Juniper was replaced with Cisco but not HP or Arista.

What if we want to find out the number of replacements in a long string?

```
>>> import re
>>> expr = '''Juniper router, HP switch, Palo Alto firewall, Juniper router, HP switch, Palo Alto firew
>>> p = re.compile('HP|Juniper|Arista')
>>> p.subn('Cisco', expr)
('Cisco router, Cisco switch, Palo Alto firewall, Cisco router, Cisco switch, Palo Alto firewall, Cisco
```

Listing 9-75. Use `subn` to count the number of replacements

You can use the `subn` method to substitute the string and find out the number of replaced strings. In [Listing 9-75](#), 13 strings have been replaced with the word *Cisco*.

Using sub and \g to Swap Positions

When you use the `sub` method in a regular expression, you can combine it with relative group referencing (`\g`). Let’s say we have a string `"Model Number : WS-C3650-48PD"`, and we want to switch the position of `'WS-C3650-48PD'` and `'Model Number'`. So, it looks like this:

From		To
Model Number : WS-C3650-48PD	→	WS-C3650-48PD : Model Number

On your Python interpreter, type in the code in each exercise. You will quickly learn how this works.

In Listing 9-76, you have compiled the match regular expression into three groups and, using the `sub` method, simply reversed the group sequence after matching each group. The group remains in the same place, but groups 1 and 3 have switched places.

```
>>> import re
>>> expr = "Model Number : WS-C3650-48PD"
>>> p = re.compile(r"(\w+\s\w+)(\s[:]\s)(\w+[-]\w+[-]\w+)")
>>> m = p.sub("\g<3>\g<2>\g<1>", expr)
>>> print(m)
WS-C3650-48PD : Model Number
Listing 9-x >>> p = re.compile(r"(?P<Desc>\w+\s\w+)(\s[:]\s)(?P<Model>(\w+[-]\w+[-]\w+))")
>>> m = p.sub("\g<Model>\g<2>\g<Desc>", expr)
>>> print(m)
WS-C3650-48PD : Model Number
Listing 9-76. Use sub and grouping to swap positions
```

In Listing 9-77, you can also apply the named grouping and give meaningful names to each group and achieve our goal. Note that the group was not given a name like `(?P<comma>\s[:]\s)` to demonstrate that you can mix-and-match the named groups and numbered groups.

```
>>> p = re.compile(r"(?P<Desc>\w+\s\w+)\s[:]\s(?P<Model>(\w+[-]\w+[-]\w+))")
>>> m = p.sub("\g<Model> : \g<Desc>", expr)
>>> print(m)
WS-C3650-48PD : Model Number
Listing 9-77. Use sub and named group method to swap positions
```

In Listing 9-77, you have only used two named groups and left the comma group out, but when you matched using `sub` method, you added the `(:)` in your expression to return the desired result.

Insert a Function in sub Method

In network operations, we use binary, decimal, and hexadecimal values. Two examples where we use hexadecimal values are MAC addresses and IPv6 addresses. This becomes most useful in IPv6 addressing scheme; it is instrumental in understanding how you can calculate from hexadecimal to binary and decimal or the other way around.

In the following exercises, we will practice functions with the `sub` method.

In Listing [9-78](#), you have used standard Python methods to convert decimal a random IP address to binary numbers. The same conversion result can be achieved by using the `sub` commands in Listing [9-79](#).

```
>>> ip = '172.168.123.245'
>>> print ('.'.join([bin(int(x)+256)[3:] for x in ip.split('.')]))
10101100.10101000.01111011.11110101
Listing 9-78. Decimal to Binary Using the join Method
```

```
>>> ip = '172.168.123.245'
>>> def dec2bin(match):
...     value = int(match.group())
...     return bin(value)
...
>>> p = re.compile(r'\d+')
>>> p.sub(dec2bin, ip)
'0b10101100.0b10101000.0b1111011.0b11110101'
Listing 9-79. Decimal to Binary Using the sub Method
```

When the `sub` method is used, the compiled return result starts with `0b`, which indicates that this is a binary number. In Listing [9-80](#), we have created a function called `dec2bin` and used it in the `sub` method to get the decimal IP address into binary numbers.

```
>>> ip = "00001010.11010110.10001011.10111101"
>>> ip1 = ip.replace(".", "")
>>> ip1
'00001010110101101000101110111101'
>>> def bin2dec():
...     return ".".join(map(str, int(ip1, 2).to_bytes(4, "big")))
...
>>> bin2dec()
'10.214.139.189'
Listing 9-80. Binary to Decimal Using Join Method
```

In Listing [9-81](#), we have selected a random binary IP address and demonstrate binary to decimal number conversion using the Python `join` method. After converting the binary number into the decimals, we know that the IP address we are trying to find is 10.214.139.189.

```
>>> mac = "84:3d:c6:f5:c9:ba"
>>> mac1 = mac.replace(":", "")
>>> mac1
'843dc6f5c9ba'
>>> i = int(mac1, 16)
>>> str(i)
'145400865868218'
Listing 9-81. Hexadecimal to Decimal Numbers
```

In Listing [9-82](#), we have captured a MAC address of a switch and want to convert this hexadecimal numbers into decimal numbers. Using the basic Python methods we can easily convert the hexadecimal number into a decimal. This is a

good example of how efficient Python could be converting numbers from one form to another.

```
>>> def hexrepl(match):
...     value = int(match.group())
...     return hex(value)
...
>>> p = re.compile(r"\d+")
>>> p.sub(hexrepl, 'MAC address: 145400865868218')
'MAC address: 0x843dc6f5c9ba'
```

Listing 9-82. Decimal to Hexadecimal

Once again, you have written a Python code with the `sub` method to reverse the decimal MAC address back into a hexadecimal number for demonstration purposes. `0x` at the beginning of the number indicates that this number is a hex number.



If you are new to regular expressions, this might have been pretty tough chapter. But as mentioned at the beginning of this chapter, in general programming, mastering the regular expressions is a must, not optional.

To get even more help on the topic, go to the following Regular Expressions Cheat Sheets.

URL: <https://cheatography.com/davechild/cheat-sheets/regular-expressions/>

URL: <http://web.mit.edu/hackl/www/lab/turkshop/slides/regex-cheatsheet.pdf>

Summary

Some people will find this chapter to be the most challenging chapter to digest. I do not expect anyone to understand this chapter in full in a single read. You will have to come back to this chapter over and over again and slowly let the regular expression syntax and concepts soak into your brain. You have just completed all the Chapter 9 exercises, and ideally, you will be able to find some spare time each week to practice regular expressions. As you write Python code, you will have to store and process more and more data. The real power in network programmability is also in the power of data processing. In the next chapter, you will be completing the necessary lab preparation and integration for Python network automation.

Table of contents collapsed